
Audio Fingerprinting

(PS1)

Working with audio files and spectrograms

Technical Notes

Facundo Franchino & Hans Dietrich

March 16, 2026

1 Objectives

Build the simplest possible audio fingerprinting prototype from scratch. The goal of this session is to load an audio file, visualise its spectrogram, and detect prominent spectral peaks as a first approximation to a song fingerprint.

2 Background

A digital audio signal is a sequence of samples taken at a fixed sample rate. By applying the Short-Time Fourier Transform (STFT), the signal can be represented as a spectrogram, showing how frequency content evolves over time. Audio fingerprinting systems such as Shazam rely on robust local peaks in this time–frequency representation.

3 Setup

We used Python 3 with `librosa`, `numpy`, `scipy`, and `matplotlib`. An MP3 file from the *Twin Peaks* soundtrack was loaded and analysed locally.

4 Tasks

4.1 Load and inspect the audio file

Load an MP3 file, inspect its sample rate, and plot the waveform in order to understand digital audio as a sequence of samples.

4.2 Generate and interpret the spectrogram

Compute the STFT, convert magnitudes to decibels, and display the spectrogram using a logarithmic frequency axis.

4.3 Detect local spectral peaks

Apply a local maximum filter and an amplitude threshold to detect salient peaks in the spectrogram, then visualise them as candidate constellation points.

5 Notes

The main practical issue was choosing suitable peak-detection parameters. A threshold that was too low produced too many peaks, especially at low frequencies. Increasing the neighbourhood size, raising the threshold, and suppressing the lowest frequency bins produced a much cleaner peak map.